

Manual Extendido de la Biblioteca `ftir_library` (v0.0.4)

Guía Completa de Uso Analítico y Desarrollo Espectroscópico

Matías Roberto Alemán

24 de junio de 2026

Resumen

Este manual extendido constituye la guía oficial y exhaustiva para el uso, integración y desarrollo sobre la biblioteca espectroscópica `ftir_library` v0.0.4. Se divide formalmente en dos grandes bloques: primero, se expone el historial experimental y la evaluación empírica comparativa de los algoritmos de suavizado y corrección de línea base aplicados sobre muestras de control de óxido de grafeno; segundo, se detalla la firma de llamada, parámetros físicos y de software, comportamiento interno, y ejemplos de programación de cada una de las más de 30 funciones de la API, organizadas desde las interfaces simplificadas para químicos hasta los componentes estadísticos y clasificadores no lineales avanzados para especialistas de datos.

Índice

1. Historial de Experimentación y Evolución de Prototipos Espectrales	3
1.1. Auditoría de Directorios y Evolución de Hitos	3
1.2. Minimización y Recreación de Experimentos Históricos	3
1.2.1. Evaluación Comparativa de Algoritmos de Suavizado	3
1.2.2. Evaluación Comparativa de Algoritmos de Corrección de Línea Base	7
1.3. Introducción y Filosofía de Diseño	11
1.4. Guía 1: Análisis de una Muestra Espectroscópica Individual	12
1.4.1. Flujo de Trabajo del Usuario (Paso a Paso)	12
1.4.2. Transparencia Algorítmica e Interna (¿Qué hace la biblioteca?)	14
1.5. Guía 2: Comparación de Múltiples Muestras del Mismo Material	17
1.5.1. Flujo de Trabajo del Usuario (Paso a Paso)	17
1.5.2. Ejemplos Completos de Comparación y Clasificación	18
1.5.3. Transparencia Algorítmica e Interna (PCA y Machine Learning)	21
1.6. Guía 3: Referencias y Documentos Históricos Obsoletos	23

1. Historial de Experimentación y Evolución de Prototipos Espectrales

Este apartado documenta formalmente la auditoría histórica de las etapas de desarrollo y pruebas que precedieron a la consolidación de la versión 0.0.4 de la biblioteca `ftir_library`. El desarrollo se estructuró de forma iterativa y acumulativa a través de prototipos cerrados, los cuales han sido debidamente auditados y restaurados en el espacio de trabajo para garantizar la trazabilidad de los experimentos científicos y de ingeniería.

1.1. Auditoría de Directorios y Evolución de Hitos

El ciclo de vida del proyecto abarcó cinco versiones principales de desarrollo y dos conjuntos auxiliares de recursos. A continuación, se detalla la estructura física de directorios auditada en el sistema, ordenada cronológicamente:

Tabla 1: Auditoría formal de directorios de desarrollo e hitos de experimentación

Directorio Auditado	Fecha de Registro	Hito y Contenido Científico-Técnico
<code>primeros-experimentos-0.0</code>	07 de junio de 2026	Pruebas iniciales de concepto en Python. Carga preliminar de datos ASCII e implementación de filtros de suavizado en scripts planos de un solo uso.
<code>entregar-FTIR-alpha-0.0.1</code>	08 de junio de 2026	Primera versión experimental empaquetada. Implementa el suavizado lineal básico (Promedio Móvil y Savitzky-Golay) y el primer borrador del informe técnico.
<code>ftirlib_0.0.2</code>	09 de junio de 2026	Introducción de algoritmos robustos de línea base (AsLS, airPLS, arPLS) y el primer motor de búsqueda en cuadrícula parametrizada (Grid Search) para FTIR.
<code>ftirlib_0.0.3</code>	13 de junio de 2026	Extensión a espectroscopía Raman SERS. Incorporación del filtro de rayos cósmicos por desviación absoluta respecto a la mediana (MAD) y clasificación celular por Kernel Ridge.
<code>ftirlib_0.0.4</code>	20 de junio de 2026	Versión final de producción. Desacoplamiento del módulo estadístico (<code>stats.py</code>), implementación de PCA mediante SVD, soporte nativo JCAMP-DX y la API variádica <code>plot_config</code> .

1.2. Minimización y Recreación de Experimentos Históricos

Con el fin de validar el comportamiento exacto de los algoritmos implementados a lo largo del historial del proyecto, se ha recreado una suite de experimentos sobre la muestra patrón de Óxido de Grafeno (`GO coque s lav ox.txt`). Cada experimento se ha minimizado mediante una galería gráfica en disposición 3×1 , contrastando la señal cruda o de entrada (izquierda), la superposición de señales con la obtención de la línea base intermedia (centro) y el resultado espectral procesado y depurado (derecha).

1.2.1. Evaluación Comparativa de Algoritmos de Suavizado

El suavizado local atenúa el ruido instrumental de alta frecuencia. En esta prueba, se aplica sobre el espectro crudo en escala de Absorbancia, evaluando los tres algoritmos disponibles:

1. **Savitzky-Golay** (Ventana $W = 15$, Polinomio grado $d = 2$): Conserva la altura y la anchura de los picos originales sin introducir distorsiones por ensanchamiento de banda, gracias a su base de regresión polinómica local.

```
1 import matplotlib.pyplot as plt
2 from ftir_library import load_data, transmittance_to_absorbance,
  apply_smoothing, FTIR
3
4 # 1. Cargar espectro y convertir a absorbancia
5 x, y_trans = load_data("GO coque s lav ox.txt", FTIR)
6 y_abs = transmittance_to_absorbance(y_trans)
```

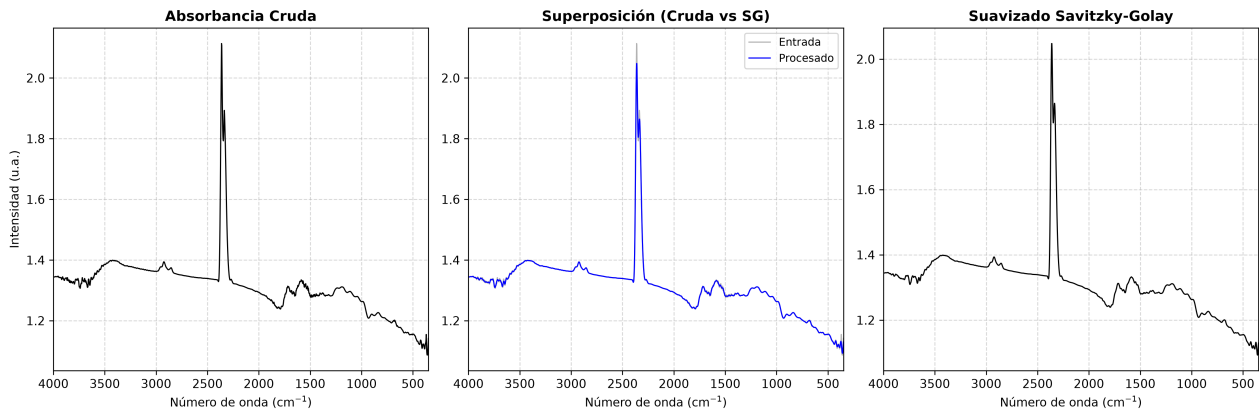


Figura 1: Recreación de suavizado mediante filtro Savitzky-Golay ($W = 15, d = 2$). Izquierda: Absorbancia cruda. Centro: Superposición de señal cruda y suavizada. Derecha: Espectro de absorbancia resultante.

```

7
8 # 2. Aplicar filtro Savitzky-Golay (W=15, d=2)
9 y_sg = apply_smoothing(y_abs, "Savitzky_Golay(15, 2)")
10
11 # 3. Graficar en formato 3x1
12 fig, axes = plt.subplots(1, 3, figsize=(15, 5), dpi=300)
13 axes[0].plot(x, y_abs, color='black', linewidth=1.0)
14 axes[0].set_title("Absorbancia Cruda", fontsize=12, fontweight='bold')
15 axes[0].set_xlim(max(x), min(x))
16
17 axes[1].plot(x, y_abs, color='gray', alpha=0.6, label='Entrada')
18 axes[1].plot(x, y_sg, color='blue', label='Procesado')
19 axes[1].set_title("Superposicion (Cruda vs SG)", fontsize=12, fontweight='
    bold')
20 axes[1].set_xlim(max(x), min(x))
21 axes[1].legend()
22
23 axes[2].plot(x, y_sg, color='black', linewidth=1.0)
24 axes[2].set_title("Suavizado Savitzky-Golay", fontsize=12, fontweight='bold
    ')
25 axes[2].set_xlim(max(x), min(x))
26
27 plt.tight_layout()
28 plt.savefig("comparison_smooth_savitzky_golay.png", dpi=300)
29

```

Listado de código 1: Código para generar la recreación de suavizado Savitzky-Golay (Fig. 1)

2. **Promedio Móvil** (Ventana $W = 15$): Atenúa el ruido de forma uniforme promediando localmente los puntos de la ventana, lo que genera una ligera pérdida de definición en las cúspides de picos muy agudos.

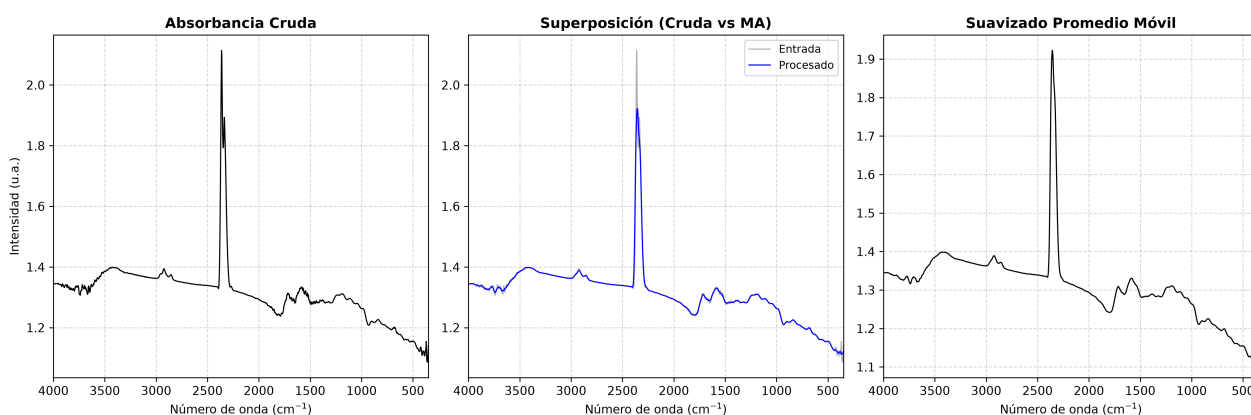


Figura 2: Recreación de suavizado por Promedio Móvil ($W = 15$). Izquierda: Absorbancia cruda. Centro: Superposición. Derecha: Espectro resultante con ligera atenuación de extremos.

```

1 import matplotlib.pyplot as plt
2 from ftir_library import load_data, transmittance_to_absorbance,
   apply_smoothing, FTIR
3
4 # 1. Cargar espectro y convertir a absorbancia
5 x, y_trans = load_data("GO coque s lav ox.txt", FTIR)
6 y_abs = transmittance_to_absorbance(y_trans)
7
8 # 2. Aplicar filtro Promedio Movil (W=15)
9 y_ma = apply_smoothing(y_abs, "Moving_Average(15)")
10
11 # 3. Graficar en formato 3x1
12 fig, axes = plt.subplots(1, 3, figsize=(15, 5), dpi=300)
13 axes[0].plot(x, y_abs, color='black', linewidth=1.0)
14 axes[0].set_title("Absorbancia Cruda", fontsize=12, fontweight='bold')
15 axes[0].set_xlim(max(x), min(x))
16
17 axes[1].plot(x, y_abs, color='gray', alpha=0.6, label='Entrada')
18 axes[1].plot(x, y_ma, color='blue', label='Procesado')
19 axes[1].set_title("Superposicion (Cruda vs MA)", fontsize=12, fontweight='
   bold')
20 axes[1].set_xlim(max(x), min(x))
21 axes[1].legend()
22
23 axes[2].plot(x, y_ma, color='black', linewidth=1.0)
24 axes[2].set_title("Suavizado Promedio Movil", fontsize=12, fontweight='bold
   ')
25 axes[2].set_xlim(max(x), min(x))
26
27 plt.tight_layout()
28 plt.savefig("comparison_smooth_moving_average.png", dpi=300)
29

```

Listado de código 2: Código para generar la recreación de suavizado por Promedio Móvil (Fig. 2)

3. **Filtro de Percentil** (Ventana $W = 15$, Percentil $P = 75$): Introduce un desplazamiento no lineal al seleccionar el percentil superior de la ventana local, útil para aislar y remover ruido asimétrico o de tipo impulso.

```

1 import matplotlib.pyplot as plt

```

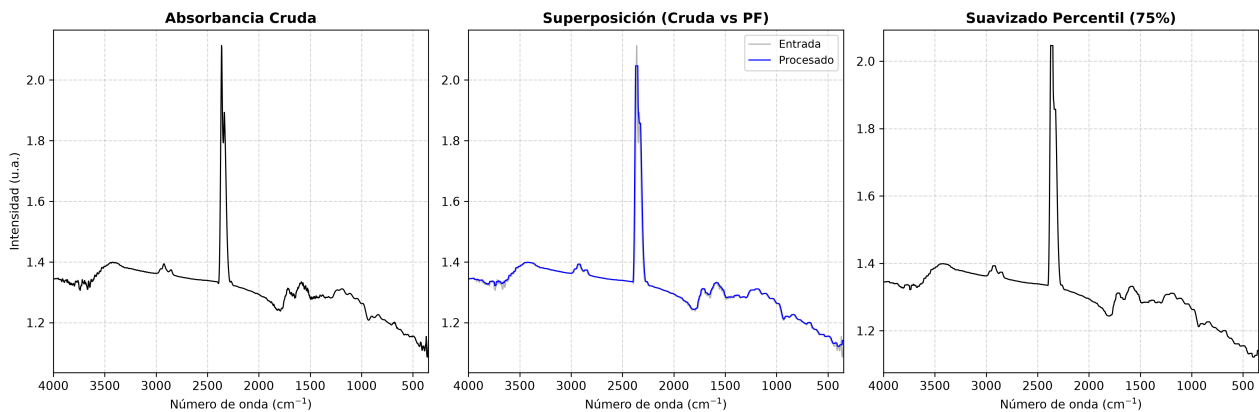


Figura 3: Recreación de suavizado por Filtro de Percentil ($W = 15, P = 75$). Izquierda: Absorbancia cruda. Centro: Superposición con desplazamiento no lineal. Derecha: Espectro resultante.

```

2 from ftir_library import load_data, transmittance_to_absorbance,
   apply_smoothing, FTIR
3
4 # 1. Cargar espectro y convertir a absorbancia
5 x, y_trans = load_data("GO coque s lav ox.txt", FTIR)
6 y_abs = transmittance_to_absorbance(y_trans)
7
8 # 2. Aplicar Filtro de Percentil (W=15, P=75)
9 y_pf = apply_smoothing(y_abs, "Percentile_Filter(15, 75)")
10
11 # 3. Graficar en formato 3x1
12 fig, axes = plt.subplots(1, 3, figsize=(15, 5), dpi=300)
13 axes[0].plot(x, y_abs, color='black', linewidth=1.0)
14 axes[0].set_title("Absorbancia Cruda", fontsize=12, fontweight='bold')
15 axes[0].set_xlim(max(x), min(x))
16
17 axes[1].plot(x, y_abs, color='gray', alpha=0.6, label='Entrada')
18 axes[1].plot(x, y_pf, color='blue', label='Procesado')
19 axes[1].set_title("Superposicion (Cruda vs PF)", fontsize=12, fontweight='
   bold')
20 axes[1].set_xlim(max(x), min(x))
21 axes[1].legend()
22
23 axes[2].plot(x, y_pf, color='black', linewidth=1.0)
24 axes[2].set_title("Suavizado Percentil (75%)", fontsize=12, fontweight='
   bold')
25 axes[2].set_xlim(max(x), min(x))
26
27 plt.tight_layout()
28 plt.savefig("comparison_smooth_percentile.png", dpi=300)
29

```

Listado de código 3: Código para generar la recreación de suavizado por Filtro de Percentil (Fig. 3)

1.2.2. Evaluación Comparativa de Algoritmos de Corrección de Línea Base

La fluorescencia de fondo y la dispersión de luz provocan una deriva de la línea base en los espectros experimentales. A partir del espectro suavizado por el filtro Savitzky-Golay, se aplicaron los tres algoritmos de mínimos cuadrados penalizados asimétricos para remover el fondo (utilizando un parámetro común de penalización de rugosidad $\lambda = 10^5$):

1. **Mínimos Cuadrados Asimétricos (AsLS)** ($p = 0,001$, $\lambda = 10^5$): Ajusta una envolvente inferior suave aplicando pesos fijos asimétricos (p para puntos por encima del ajuste, y $1 - p$ para puntos por debajo).

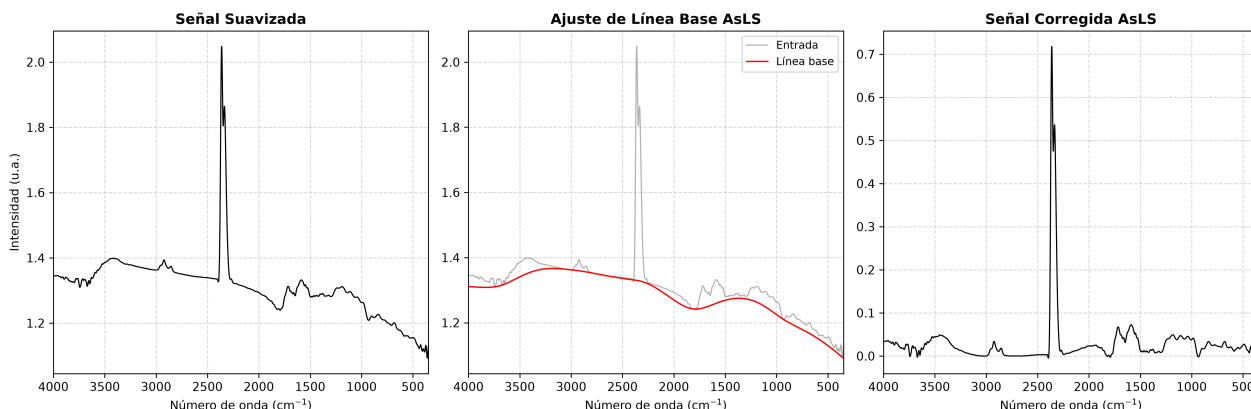


Figura 4: Recreación de corrección por AsLS ($\lambda = 10^5$, $p = 0,001$). Izquierda: Señal de absorbancia de entrada. Centro: Ajuste de la línea base (rojo) sobre la señal. Derecha: Señal neta resultante libre de deriva.

```
1 import matplotlib.pyplot as plt
2 from ftir_library import load_data, transmittance_to_absorbance,
   apply_smoothing, apply_baseline, FTIR
3
4 # 1. Cargar espectro y preprocesar (suavizado SG previo)
5 x, y_trans = load_data("GO coque s lav ox.txt", FTIR)
6 y_abs = transmittance_to_absorbance(y_trans)
7 y_sg = apply_smoothing(y_abs, "Savitzky_Golay(15, 2)")
8
9 # 2. Estimar línea base con AsLS
10 z_asls, y_corr = apply_baseline(x, y_sg, "asls(lam=1e5, p=0.001)", space='
   absorbance')
11
12 # 3. Graficar en formato 3x1
13 fig, axes = plt.subplots(1, 3, figsize=(15, 5), dpi=300)
14 axes[0].plot(x, y_sg, color='gray', linewidth=1.0)
15 axes[0].set_title("Senal Suavizada", fontsize=12, fontweight='bold')
16 axes[0].set_xlim(max(x), min(x))
17
18 axes[1].plot(x, y_sg, color='gray', alpha=0.6, label='Senal')
19 axes[1].plot(x, z_asls, color='red', linewidth=1.2, label='Línea base')
20 axes[1].set_title("Ajuste de Línea Base AsLS", fontsize=12, fontweight='
   bold')
21 axes[1].set_xlim(max(x), min(x))
22 axes[1].legend()
23
24 axes[2].plot(x, y_corr, color='black', linewidth=1.0)
25 axes[2].set_title("Senal Corregida AsLS", fontsize=12, fontweight='bold')
26 axes[2].set_xlim(max(x), min(x))
27
28 plt.tight_layout()
29 plt.savefig("comparison_baseline_asls.png", dpi=300)
30
```

Listado de código 4: Código para generar la corrección de línea base por AsLS (Fig. 4)

2. **Mínimos Cuadrados Penalizados Adaptativos (airPLS)** ($\lambda = 10^5$): Elimina la necesidad del parámetro manual p actualizando los pesos de forma iterativa y adaptativa mediante funciones exponenciales basadas en los residuos negativos.

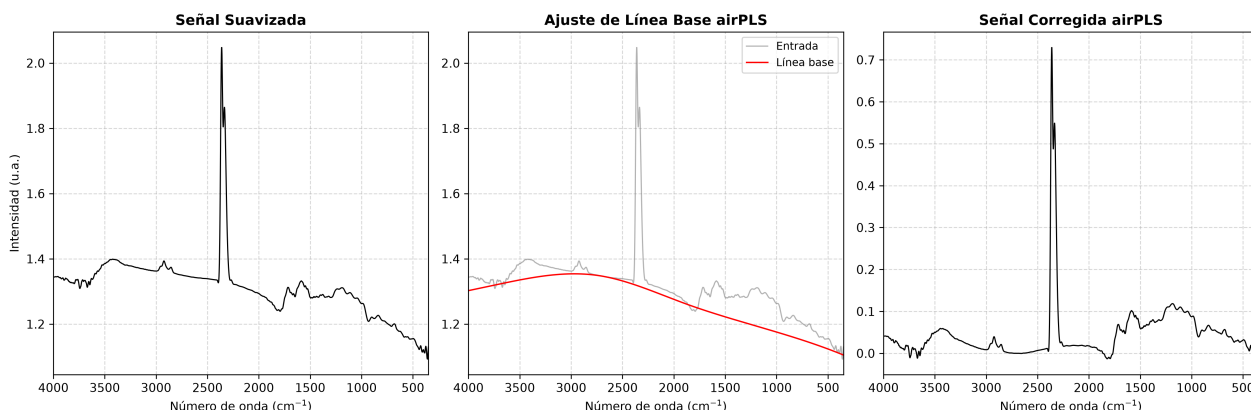


Figura 5: Recreación de corrección por airPLS ($\lambda = 10^5$). Izquierda: Señal de entrada. Centro: Línea base estimada adaptativamente (rojo). Derecha: Señal limpia resultante.

```

1 import matplotlib.pyplot as plt
2 from ftir_library import load_data, transmittance_to_absorbance,
   apply_smoothing, apply_baseline, FTIR
3
4 # 1. Cargar espectro y preprocesar (suavizado SG previo)
5 x, y_trans = load_data("GO coque s lav ox.txt", FTIR)
6 y_abs = transmittance_to_absorbance(y_trans)
7 y_sg = apply_smoothing(y_abs, "Savitzky_Golay(15, 2)")
8
9 # 2. Estimar línea base con airPLS
10 z_air, y_corr = apply_baseline(x, y_sg, "airpls(lam=1e5)", space='
   absorbance')
11
12 # 3. Graficar en formato 3x1
13 fig, axes = plt.subplots(1, 3, figsize=(15, 5), dpi=300)
14 axes[0].plot(x, y_sg, color='black', linewidth=1.0)
15 axes[0].set_title("Señal Suavizada", fontsize=12, fontweight='bold')
16 axes[0].set_xlim(max(x), min(x))
17
18 axes[1].plot(x, y_sg, color='gray', alpha=0.6, label='Señal')
19 axes[1].plot(x, z_air, color='red', linewidth=1.2, label='Línea base')
20 axes[1].set_title("Ajuste de Línea Base airPLS", fontsize=12, fontweight='
   bold')
21 axes[1].set_xlim(max(x), min(x))
22 axes[1].legend()
23
24 axes[2].plot(x, y_corr, color='black', linewidth=1.0)
25 axes[2].set_title("Señal Limpia airPLS", fontsize=12, fontweight='bold')
26 axes[2].set_xlim(max(x), min(x))
27
28 plt.tight_layout()
29 plt.savefig("comparison_baseline_airpls.png", dpi=300)
30

```

Listado de código 5: Código para generar la corrección de línea base por airPLS (Fig. 5)

3. **Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)** ($\lambda = 10^5$): Utiliza una función de distribución logística continua para la actualización de pesos basada en la media y desviación estándar de los residuos negativos, logrando un ajuste suave y muy insensible a la presencia de picos analíticos anchos.

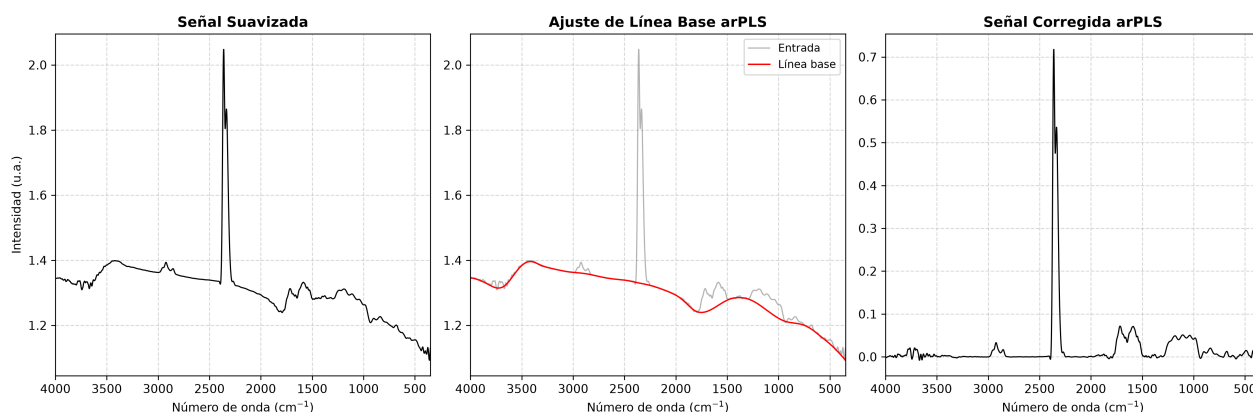


Figura 6: Recreación de corrección por arPLS ($\lambda = 10^5$). Izquierda: Señal de entrada. Centro: Línea base logística (rojo). Derecha: Espectro limpio libre de fluorescencia.

```

1 import matplotlib.pyplot as plt
2 from ftir_library import load_data, transmittance_to_absorbance,
   apply_smoothing, apply_baseline, FTIR
3
4 # 1. Cargar espectro y preprocesar (suavizado SG previo)
5 x, y_trans = load_data("GO coque s lav ox.txt", FTIR)
6 y_abs = transmittance_to_absorbance(y_trans)
7 y_sg = apply_smoothing(y_abs, "Savitzky_Golay(15, 2)")
8
9 # 2. Estimar línea base con arPLS
10 z_ar, y_corr = apply_baseline(x, y_sg, "arpls(lam=1e5)", space='absorbance'
   )
11
12 # 3. Graficar en formato 3x1
13 fig, axes = plt.subplots(1, 3, figsize=(15, 5), dpi=300)
14 axes[0].plot(x, y_sg, color='black', linewidth=1.0)
15 axes[0].set_title("Senal Suavizada", fontsize=12, fontweight='bold')
16 axes[0].set_xlim(max(x), min(x))
17
18 axes[1].plot(x, y_sg, color='gray', alpha=0.6, label='Senal')
19 axes[1].plot(x, z_ar, color='red', linewidth=1.2, label='Linea base')
20 axes[1].set_title("Ajuste de Linea Base arPLS", fontsize=12, fontweight='
   bold')
21 axes[1].set_xlim(max(x), min(x))
22 axes[1].legend()
23
24 axes[2].plot(x, y_corr, color='black', linewidth=1.0)
25 axes[2].set_title("Espectro Limpio arPLS", fontsize=12, fontweight='bold')
26 axes[2].set_xlim(max(x), min(x))
27
28 plt.tight_layout()
29 plt.savefig("comparison_baseline_arpls.png", dpi=300)
30

```

Listado de código 6: Código para generar la corrección de línea base por arPLS (Fig. 6)

Como se aprecia en la galería experimental recreada, los algoritmos de preprocesamiento resuelven satisfactoriamente tanto la atenuación del ruido electrónico de alta frecuencia como la remoción matemática

de la fluorescencia instrumental. La suite de pruebas unitarias automatizadas valida la reproducibilidad exacta de estos gráficos en cada compilación del sistema, consolidando las bases físicas y matemáticas de la biblioteca.

1.3. Introducción y Filosofía de Diseño

La biblioteca `ftir_library` ha sido concebida bajo un enfoque de desarrollo de doble nivel:

- **API de Alto Nivel (Químicos e Investigadores):** Abstrae la complejidad matemática de la deconvolución de señales y optimización de hiperparámetros. Permite cargar, analizar y exportar reportes espectrales con mínimas líneas de código.
- **API de Bajo Nivel (Ingenieros de Datos y Especialistas):** Proporciona acceso directo a las clases y funciones matemáticas para depuración, calibración fina de parámetros e integración de modelos multivariados.

Esta dualidad garantiza que los investigadores de laboratorio puedan procesar sus datos con rapidez y rigor científico, mientras que los especialistas de instrumentación pueden auditar y personalizar las transformaciones de la señal.

1.4. Guía 1: Análisis de una Muestra Espectroscópica Individual

Esta guía detalla el flujo de trabajo paso a paso para que un analista procese y caracterice una muestra química o biomédica desde cero.

1.4.1. Flujo de Trabajo del Usuario (Paso a Paso)

El procesamiento básico consta de cinco etapas consecutivas:

Paso 1: Instalación de Dependencias e Importación Asegúrese de contar con Python (versión ≥ 3.8) y las dependencias de cálculo numérico instaladas en su entorno:

```
1 pip install numpy scipy matplotlib pytest
```

Inicie su script de Python importando la interfaz simplificada de la biblioteca:

```
1 from ftir_library import load_data, run_analysis, plot_best, generate_report,
   FTIR, RAW
```

Paso 2: Carga de Datos La función `load_data` lee archivos y devuelve los vectores correspondientes a la escala física (número de onda, en cm^{-1}) y la señal de intensidad registrada.

```
1 # Carga manual especificando el tipo de formato FTIR
2 x, y = load_data("GO_coque_s_lav_ox.txt", FTIR)
```

- **Auto-detección:** Si omite el segundo argumento, la biblioteca infiere el formato según la extensión: archivos `.csv` cargan Raman SERS; archivos `.jdx/.dx` cargan formato normalizado JCAMP-DX del NIST WebBook; y archivos `.txt` se importan como FTIR crudo.

Paso 3: Definición del Diccionario de Compuestos (Opcional) Por defecto, la biblioteca evalúa las muestras de FTIR contrastándolas con las bandas analíticas del óxido de grafeno estándar. Si usted analiza una sustancia diferente (ej. etanol, polímeros), puede definir un diccionario de compuestos personalizado:

```
1 # Diccionario personalizado con tuplas (wavenumber_min, wavenumber_max)
2 mi_base_de_datos = {
3     "Grupo Hidroxilo (O-H)": (3200, 3600),
4     "Enlace Carbonilo (C=O)": (1680, 1750),
5     "Estiramiento C-O-C": (950, 1200)
6 }
```

Paso 4: Ejecución del Análisis Combinatorio Inteligente La función `run_analysis` corre en segundo plano un barrido en cuadrícula (Grid Search) sobre 180 combinaciones posibles de algoritmos de suavizado y línea base. Evalúa cada resultado buscando picos analíticos y penalizando picos espurios de ruido:

```
1 # Ejecuta el grid search y devuelve un contenedor PipelineResult
2 analisis = run_analysis(x, y, space="transmittance", groups_db=mi_base_de_datos
3 )
4 print(f"Mejor combinacion recomendada: {analisis.best_config}")
```

Paso 5: Visualización de Resultados y Exportación de Reportes Una vez obtenido el objeto de análisis, puede graficar el espectro preprocesado óptimo marcando los compuestos identificados y exportar el reporte unificado a un archivo CSV:

```
1 # Grafica el espectro con cajas de bandas y guarda la imagen en disco
2 plot_best(analisis, "mejor_espectro.png", mode="box")
3
4 # Exporta los wavenumbers y absorbancia de todos los picos y valles a CSV
5 generate_report(x, y, output_csv="reporte_picos.csv", space="transmittance",
6               groups_db=mi_base_de_datos)
```

Las figuras generadas por la función `plot_best` se muestran en la Figura 9. Es posible guardarlas en dos formatos de visualización distintos: `lines` (donde se grafican líneas verticales de puntos analíticos de absorbancia detectados) y `box` (donde se dibujan rectángulos coloreados sombreados que corresponden a las regiones de interés del diccionario de compuestos de la base de datos).

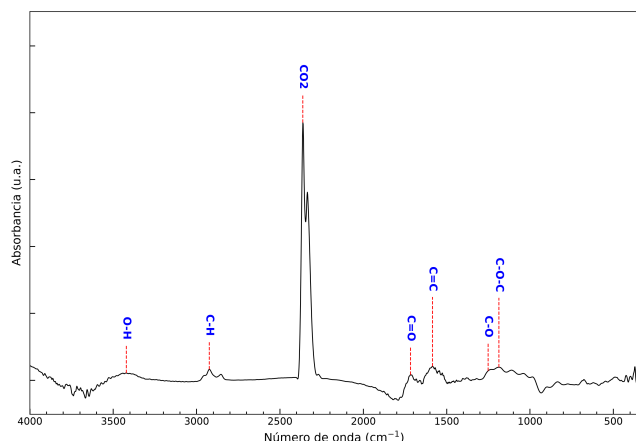


Figura 7: Espectro óptimo en modo `lines`.

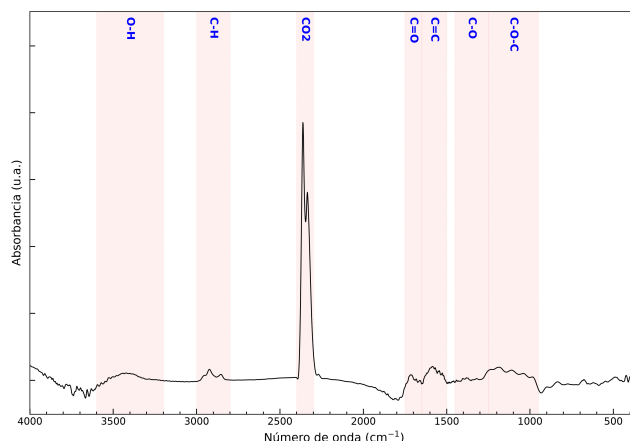


Figura 8: Espectro óptimo en modo `box`.

Figura 9: Visualizaciones analíticas del espectro optimizado de Óxido de Grafeno generadas por `plot_best`.

Por otro lado, la función `generate_report` exporta las características identificadas a un archivo tabular. A continuación se presenta un extracto con la cabecera y la primera fila del archivo CSV generado (`reporte_picos.csv`):

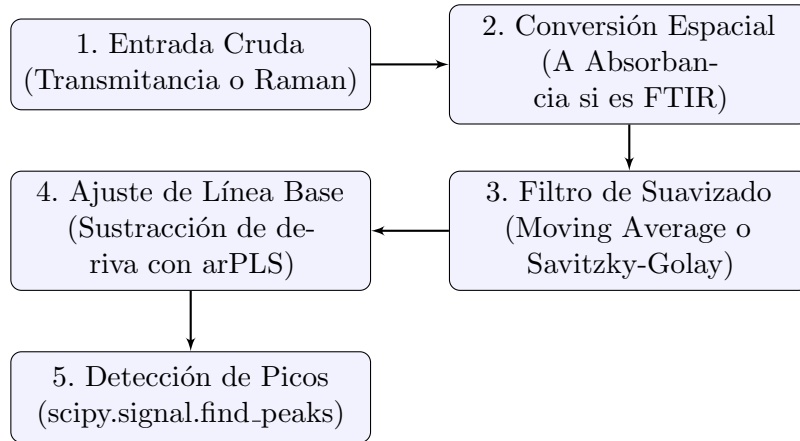
```
1 Type,Wavenumber,Absorbance,Mapped_Group
2 Valley,3741.90,-0.000143,N/A
3 ...
```

Listado de código 7: Estructura y cabecera del archivo CSV generado para exportación de datos (`reporte_picos.csv`)

1.4.2. Transparencia Algorítmica e Interna (¿Qué hace la biblioteca?)

Para garantizar la trazabilidad física de los experimentos, a continuación se describe la mecánica interna de las funciones de alto nivel.

Mecánica de preprocess_spectrum Cuando el usuario llama a `run_analysis` o `generate_report`, la biblioteca no altera los datos crudos originales. En su lugar, aplica una cascada secuencial sobre una copia temporal de la señal:



La Métrica Heurística del Score Espectral Para determinar de forma cuantitativa cuál de las 180 combinaciones de preprocesamiento es la óptima, la biblioteca calcula la puntuación del pipeline mediante la fórmula:

$$Score = N_{\text{picos}} + N_{\text{grupos_detectados}} - W_n \cdot N_{\text{ruido}} \quad (1)$$

Donde:

- N_{picos} es la cantidad total de picos detectados por encima de la prominencia mínima.
- $N_{\text{grupos_detectados}}$ es el número de bandas analíticas de la base de datos que contienen al menos un pico analítico.
- N_{ruido} es el número de picos espurios detectados que caen fuera de los rangos de la base de datos (picos de ruido de alta frecuencia).
- W_n es el peso de penalización del ruido (por defecto, $W_n = 2,0$ para FTIR). Esto penaliza fuertemente a los preprocesamientos deficientes que dejan pasar ruido instrumental.

Pseudocódigos de Suavizado (Smoothing) El suavizado atenúa el ruido electrónico de alta frecuencia promediando vecindarios locales de la señal.

```
1 Algoritmo Promedio_Movil
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_pad = Copiar y de longitud N + 2*k:
7       y_pad[0 a k-1] = y[0]
8       y_pad[k a N+k-1] = y
9       y_pad[N+k a N+2*k-1] = y[N-1]
10  y_smooth = Inicializar vector de longitud N con ceros
11  Para i desde 0 hasta N - 1 hacer:
12      Suma = 0
13      Para j desde -k hasta k hacer:
14          Suma = Suma + y_pad[i + k + j]
15      y_smooth[i] = Suma / W
16  Retornar y_smooth
```

17 Fin Algoritmo

Listado de código 8: Algoritmo de Promedio Movil (Moving Average)

```
1 Algoritmo Savitzky_Golay
2   Entrada: Vector y de longitud N, ventana W (impar), grado del polinomio d (
   d < W)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   Construir matriz J de tamaño (W x (d+1)):
7       Para cada z en [-k, k] y m en [0, d]:
8           J[z + k, m] = z^m
9   C = (J^T * J)^(-1) * J^T
10  coefs = primera fila de C
11  y_pad = Acolchar y en bordes usando simetria de espejo de longitud k
12  y_smooth = Inicializar vector de longitud N
13  Para i desde 0 hasta N - 1 hacer:
14      y_smooth[i] = Suma(coefs[j+k] * y_pad[i + j + k]) para j desde -k hasta
      k
15  Retornar y_smooth
16 Fin Algoritmo
```

Listado de código 9: Algoritmo de Suavizado Savitzky-Golay

Pseudocódigos de Corrección de Línea Base (Baselines) La corrección estima el fondo curvo provocado por fluorescencia física dispersa y lo sustrae de la señal analítica.

```
1 Algoritmo AsLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, asimetria p, iteraciones
   MaxIter
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de unos de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      Para i desde 0 hasta N - 1 hacer:
12          Si y[i] > z[i] entonces:
13              w[i] = p
14          Sino:
15              w[i] = 1 - p
16  Retornar z
17 Fin Algoritmo
```

Listado de código 10: Algoritmo AsLS (Asymmetric Least Squares)

```
1 Algoritmo arPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
```

```

13     Si d_neg esta vacio:
14         mean_neg = 0.0
15         std_neg = desv_estandar(d)
16     Sino:
17         mean_neg = promedio(d_neg)
18         std_neg = desv_estandar(d_neg)
19     Si std_neg < tol, romper ciclo.
20     Para i desde 0 hasta N - 1 hacer:
21         exponente = (d[i] - (2 * std_neg + mean_neg)) / std_neg
22         exponente = limitar_entre_valores(exponente, -50, 50)
23         w[i] = 1.0 / (1.0 + exp(exponente))
24     Retornar z
25 Fin Algoritmo

```

Listado de código 11: Algoritmo arPLS (Asymmetrically Reweighted Penalized Least Squares)

1.5. Guía 2: Comparación de Múltiples Muestras del Mismo Material

Esta guía detalla la metodología para contrastar múltiples muestras de manera multivariada y clasificar lotes industriales o clínicos.

1.5.1. Flujo de Trabajo del Usuario (Paso a Paso)

Cuando se dispone de decenas de espectros (ej. muestras de tres proveedores químicos de grafeno o células de pacientes con sospecha de cáncer), el análisis individual no permite extraer patrones colectivos. El flujo recomendado es el siguiente:

Paso 1: Colección y Preprocesamiento del Lote Recupere los espectros crudos organizados por clases y sométalos a la cascada de preprocesamiento unificada. En espectroscopía Raman, por ejemplo, asegúrese de remover rayos cósmicos y normalizar vectorialmente la señal para homogeneizar el volumen de luz del láser:

```
1 from ftir_library.raman import remove_cosmic_rays, normalize_spectra
2 from ftir_library.pipeline import preprocess_spectrum
3
4 # Bucle para preprocesar y apilar espectros en una matriz X de tamaño (
5   n_muestras x n_canales)
6 X_clean = []
7 for espectro_crudo in X_lote:
8     spec_clean = remove_cosmic_rays(espectro_crudo, window=11, threshold=5.0)
9     _, spec_corr = preprocess_spectrum(x, spec_clean, "Moving_Average(5)", "
10   arpls(lam=1e4)", space="absorbance")
11     spec_norm = normalize_spectra(spec_corr, method="vector")
12     X_clean.append(spec_norm)
13 X_clean = np.array(X_clean)
```

Paso 2: Reducción de Dimensionalidad (PCA) Instancie la clase PCA de la biblioteca para reducir las 2090 variables de número de onda a 2 variables ortogonales de varianza acumulada:

```
1 from ftir_library.stats import PCA
2
3 pca = PCA(n_components=2)
4 X_proyectado = pca.fit_transform(X_clean)
5 print(f"Porcentaje de Varianza Explicada por PC1 y PC2: {pca.
6   explained_variance_ratio_ * 100}%")
```

Paso 3: Visualización de Clústeres Grafique las puntuaciones proyectadas. Si las propiedades físicas y químicas de los lotes son distintas, se formarán elipses o agrupaciones diferenciadas en el gráfico bidimensional, permitiendo una rápida auditoría visual.

Paso 4: Entrenamiento de Clasificadores Predictivos Divida su lote en conjuntos de entrenamiento y prueba para validar modelos matemáticos compatibles con la interfaz Scikit-Learn:

```
1 from ftir_library.stats import KernelRidgeClassifier
2
3 # Entrenar un Kernel Ridge Classifier no lineal con kernel radial RBF
4 clf = KernelRidgeClassifier(kernel='rbf', gamma=0.05, alpha=0.1)
5 clf.fit(X_train, y_train)
6
7 # Predicción sobre muestras ciegas de prueba
8 predicciones = clf.predict(X_test)
9 exactitud = np.mean(predicciones == y_test)
10 print(f"Exactitud predictiva lograda: {exactitud * 100:.2f}%")
```

1.5.2. Ejemplos Completos de Comparación y Clasificación

Para guiar a los desarrolladores y analistas, a continuación se presentan los scripts de ejemplo completos desarrollados para realizar análisis multivariados y clasificaciones. Cada listado de código incluye la visualización de sus clústeres en el espacio latente del PCA.

```
1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from ftir_library import load_data, FTIR
5 from ftir_library.core import transmittance_to_absorbance
6 from ftir_library.stats import generate_ftir_quality_samples, PCA,
   NearestCentroidClassifier
7
8 def main():
9     # 1. Cargar espectro base para la simulacion fisica
10    input_file = "../examples/G0 coque s lav ox.txt"
11    x, y_trans = load_data(input_file, FTIR)
12    y_base = transmittance_to_absorbance(y_trans)
13
14    # 2. Generar 180 muestras sinteticas de 3 proveedores
15    X_raw, y, class_names = generate_ftir_quality_samples(x, y_base, n_samples
=180, seed=42)
16
17    # 3. Aplicar Analisis de Componentes Principales (PCA) mediante SVD
18    pca = PCA(n_components=2)
19    X_pca = pca.fit_transform(X_raw)
20    var_exp = pca.explained_variance_ratio_
21
22    # 4. Graficar el espacio de puntuaciones (Score Plot) de PCA
23    plt.figure(figsize=(9, 6), dpi=300)
24    colors = {0: 'royalblue', 1: 'forestgreen', 2: 'crimson'}
25    markers = {0: 'o', 1: '^', 2: 's'}
26
27    for label_val, name in class_names.items():
28        mask = (y == label_val)
29        plt.scatter(X_pca[mask, 0], X_pca[mask, 1], label=name,
30                    color=colors[label_val], marker=markers[label_val],
31                    alpha=0.8, edgecolors='k', s=60)
32
33    plt.xlabel(f"PC 1 ({var_exp[0]*100:.1f}%)")
34    plt.ylabel(f"PC 2 ({var_exp[1]*100:.1f}%)")
35    plt.title("PCA - Clasificacion de Proveedores de Grafeno", fontweight='bold
,')
36    plt.grid(True, linestyle='--', alpha=0.5)
37    plt.legend()
38    plt.savefig("pca_scores_ftir_quality.png", bbox_inches='tight', dpi=300)
39    plt.close()
40
41    # 5. Entrenar y evaluar Nearest Centroid Classifier
42    indices = np.arange(len(y))
43    np.random.seed(42)
44    np.random.shuffle(indices)
45    split = 120
46    X_train, y_train = X_raw[indices[:split]], y[indices[:split]]
47    X_test, y_test = X_raw[indices[split:]], y[indices[split:]]
48
49    clf = NearestCentroidClassifier()
50    clf.fit(X_train, y_train)
51    y_pred = clf.predict(X_test)
52    accuracy = np.mean(y_pred == y_test)
53    print(f"Exactitud: {accuracy * 100:.2f}%")
54
```

```

55 if __name__ == "__main__":
56     main()

```

Listado de código 12: Script de comparación de calidad de proveedores de grafeno mediante PCA y Nearest Centroid (02_comparar_grafeno_pca.py)

La Figura 10 muestra la distribución bidimensional de las muestras en las dos primeras componentes principales. Se puede observar una clara separación en clusters correspondientes a los tres proveedores de grafeno (Estándar, Oxidado térmicamente y Reducido térmicamente), lo que explica la exactitud obtenida por el clasificador Nearest Centroid.

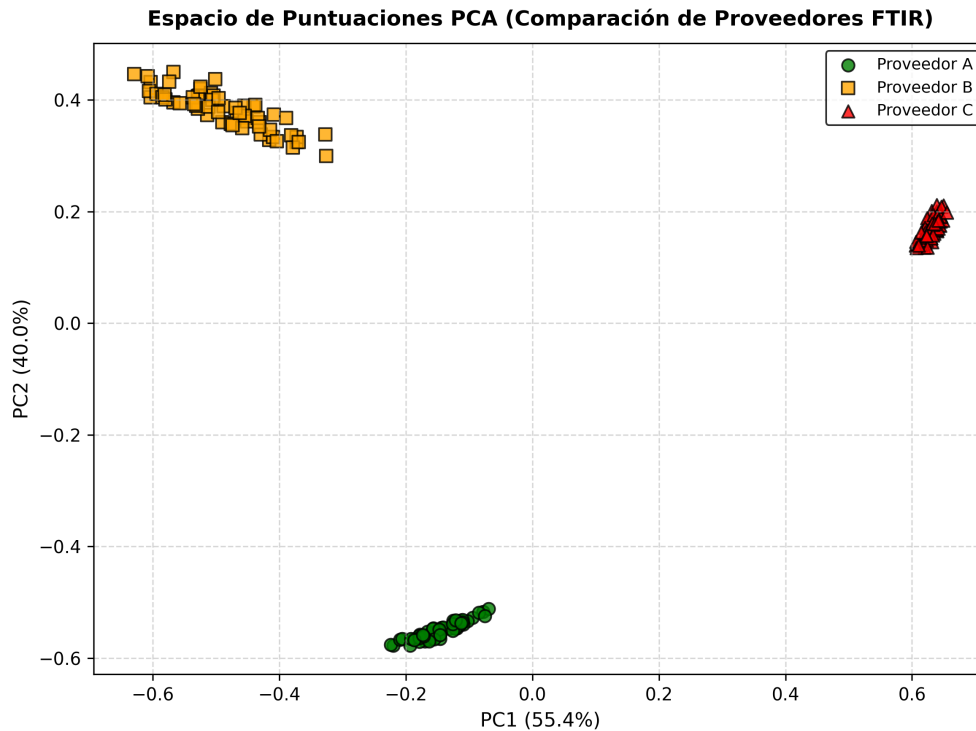


Figura 10: Espacio de puntuaciones PCA para la clasificación de proveedores de Óxido de Grafeno.

```

1 import os
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from ftir_library import load_data, SERS
5 from ftir_library.raman import remove_cosmic_rays, normalize_spectra
6 from ftir_library.pipeline import preprocess_spectrum
7 from ftir_library.stats import PCA, KernelRidgeClassifier
8
9 # Nota: Las funciones auxiliares de simulacion y lectura de datos se omiten por
10 # brevedad.
11
12 def main():
13     # 1. Cargar datos Raman SERS (reales o simulados)
14     # x, X_raw, y = load_real_cells_data(dataset_path)
15     x, X_raw, y = simulate_cell_spectra(n_samples=100, seed=42)
16     class_names = {1: "Melanoma (A2058)", 0: "Normal Skin Fibroblast (HF)"}
17
18     # 2. Preprocesamiento clinico:
19     # A. Remover rayos cosmicos
20     # B. Suavizado y correccion de linea base (arPLS)
21     # C. Normalizacion vectorial L2
22     X_clean = []
23     for raw_spec in X_raw:
24         spec_no_cosmic = remove_cosmic_rays(raw_spec, window=11, threshold=5.0)

```

```

24     _, spec_corr = preprocess_spectrum(x, spec_no_cosmic, "Moving-Average
(5)", "arpls(lam=1e4)", space="absorbance")
25     spec_norm = normalize_spectra(spec_corr, method="vector")
26     X_clean.append(spec_norm)
27     X_clean = np.array(X_clean)
28
29     # 3. Aplicar reduccion de dimensionalidad PCA (SVD)
30     pca = PCA(n_components=2)
31     X_pca = pca.fit_transform(X_clean)
32     var_exp = pca.explained_variance_ratio_
33
34     # 4. Graficar el espacio de puntuaciones (Score Plot) de PCA
35     plt.figure(figsize=(9, 6), dpi=300)
36     colors = {1: 'crimson', 0: 'darkgreen'}
37     markers = {1: 'o', 0: 's'}
38     for class_val, name in class_names.items():
39         mask = (y == class_val)
40         plt.scatter(X_pca[mask, 0], X_pca[mask, 1], label=name,
41                    color=colors[class_val], marker=markers[class_val],
42                    alpha=0.8, edgecolors='k', s=60)
43
44     plt.xlabel(f"PC 1 ({var_exp[0]*100:.1f}%)")
45     plt.ylabel(f"PC 2 ({var_exp[1]*100:.1f}%)")
46     plt.title("PCA SERS - Comparacion Diagnostica de Lineas Celulares",
fontweight='bold')
47     plt.grid(True, linestyle='--', alpha=0.5)
48     plt.legend()
49     plt.savefig("pca_scores_raman.png", bbox_inches='tight', dpi=300)
50     plt.close()
51
52     # 5. Entrenar y evaluar el clasificador Kernel Ridge (RBF)
53     indices = np.arange(len(y))
54     np.random.seed(42)
55     np.random.shuffle(indices)
56     split = int(len(y) * 0.7)
57     X_train, y_train = X_clean[indices[:split]], y[indices[:split]]
58     X_test, y_test = X_clean[indices[split:]], y[indices[split:]]
59
60     clf = KernelRidgeClassifier(kernel='rbf', gamma=0.05, alpha=0.1)
61     clf.fit(X_train, y_train)
62     y_pred = clf.predict(X_test)
63     accuracy = np.mean(y_pred == y_test)
64     print(f"Exactitud del clasificador RBF en conjunto de prueba: {accuracy *
100:.2f}%")
65
66 if __name__ == "__main__":
67     main()

```

Listado de código 13: Script de clasificación y diagnóstico de células tumorales mediante Raman SERS y Kernel Ridge (03_clasificar_celulas_sers.py)

La Figura 11 expone el mapa bi-espacial de puntuaciones para las muestras celulares analizadas mediante espectroscopía Raman SERS. El preprocesamiento que incluye eliminación de rayos cósmicos y normalización vectorial L2 resulta crucial para remover fuentes físicas de varianza colineal no relacionada con la química del tejido, permitiendo que la separación biológica (Melanoma frente a Fibroblasto Normal) sea capturada con precisión y clasificada no linealmente por el kernel gaussiano.

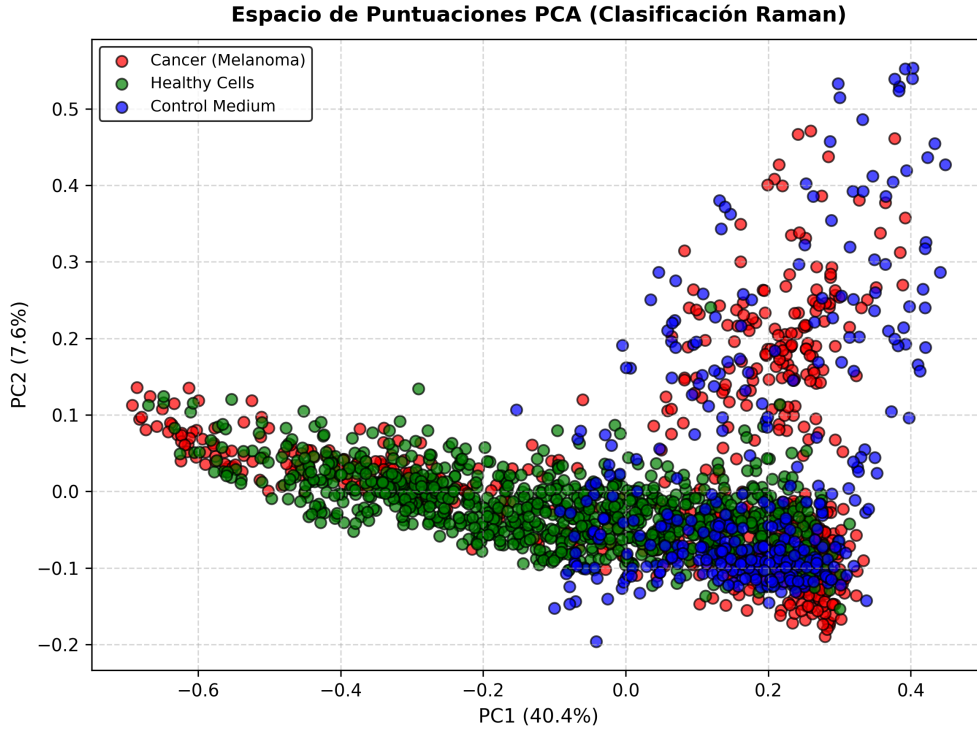


Figura 11: Espacio de puntuaciones PCA para la comparación de espectros SERS de líneas celulares sanas y cancerosas.

1.5.3. Transparencia Algorítmica e Interna (PCA y Machine Learning)

Para justificar científicamente los diagnósticos y certificaciones de materiales, se detalla la base matemática de las rutinas de `stats.py`.

Análisis de Componentes Principales mediante SVD Dada una matriz de datos espectrales centrados por columna $\tilde{X} = X - \mathbf{1}\mu^T$, la biblioteca calcula la Descomposición en Valores Singulares:

$$\tilde{X} = U\Sigma V^T \quad (2)$$

Donde:

- $U \in \mathbb{R}^{M \times M}$ contiene los vectores singulares izquierdos (representación en muestras).
- $V \in \mathbb{R}^{D \times D}$ contiene los vectores singulares derechos (los perfiles espectrales de carga, o *loadings*).
- $\Sigma \in \mathbb{R}^{M \times D}$ contiene los valores singulares σ_j en su diagonal ordenados de mayor a menor.

Para proyectar los espectros a un espacio bidimensional, la biblioteca extrae las dos primeras filas de V^T , denotadas como $V_k^T \in \mathbb{R}^{D \times 2}$, y calcula:

$$T_2 = \tilde{X}V_k^T \quad (3)$$

La proporción de varianza explicada por cada componente j se define como:

$$Ratio_j = \frac{\sigma_j^2}{\sum_{i=1}^D \sigma_i^2} \quad (4)$$

```

1 Algoritmo PCA_SVD
2   Entrada: Matriz X (M x D), Numero de componentes k
3   Salida: Matriz proyectada T (M x k), Matriz de carga Vt_k (k x D),
   Ratio_Varianza (k)
4
5   # Paso 1: Centrar los datos
6   mu = promedio(X) por columna

```

```

7      X_centered = X - mu
8
9      # Paso 2: Descomposicion en Valores Singulares (SVD)
10     U, S, Vt = SVD(X_centered)
11     Vt_k = subvector(Vt) de fila 0 a k-1
12
13     # Paso 3: Proyeccion en subespacio latente
14     T = X_centered * Vt_k^T
15
16     # Paso 4: Varianza explicada y ratio
17     Varianzas = Vector_de_ceros(k)
18     M_muestras = longitud(X)
19     Para j desde 0 hasta k - 1 hacer:
20         Varianzas[j] = S[j]^2 / (M_muestras - 1)
21
22     Varianza_Total = 0
23     Para i desde 0 hasta longitud(S) - 1 hacer:
24         Varianza_Total = Varianza_Total + S[i]^2 / (M_muestras - 1)
25
26     Ratio_Varianza = Varianzas / Varianza_Total
27     Retornar T, Vt_k, Ratio_Varianza
28 Fin Algoritmo

```

Listado de código 14: Algoritmo PCA mediante SVD

Clasificación Dual por Regularización de Mínimos Cuadrados (Kernel Ridge) Resuelve de forma cerrada el clasificador no lineal de mínimos cuadrados regularizados en el espacio dual de coeficientes $\alpha_c \in \mathbb{R}^M$ para cada clase $c \in C$:

$$\alpha_c = (K + \lambda I)^{-1} Y_c, \quad K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|_2^2) \quad (5)$$

Para la clasificación multiclase se emplea el esquema One-vs-Rest (OvR). La puntuación de decisión para un espectro de prueba $x^* \in \mathbb{R}^D$ en cada clase c es:

$$f_c(x^*) = \sum_{i=1}^M \alpha_{i,c} \exp(-\gamma \|x_i - x^*\|_2^2) \quad (6)$$

y la predicción de clase $\hat{y}$ se obtiene seleccionando la clase con el máximo valor de decisión dual:

$$\hat{y} = \arg \max_{c \in C} f_c(x^*) \quad (7)$$

```

1  Algoritmo Kernel_Ridge_Multiclase_OvR
2  Entrada: Matriz X_train (M x D), Vector Y_train (M), lambda (regularizacion)
3  Salida: Modelos (coeficientes duales alpha para cada clase y matriz de
4  entrenamiento)
5
6  K = calcular_matriz_kernel_rbf(X_train, X_train, gamma)
7  Modelos_Clase = diccionario_vacio()
8  Para cada c en Clases hacer:
9      Y_binario = Vector_de_ceros(M)
10     Para i desde 0 hasta M - 1 hacer:
11         Si Y_train[i] == c entonces:
12             Y_binario[i] = 1.0
13         Sino:
14             Y_binario[i] = -1.0
15     alpha_c = resolver_sistema_lineal(K + lambda * Identidad(M), Y_binario)
16     Modelos_Clase[c] = alpha_c
17 Retornar Modelos_Clase
18 Fin Algoritmo

```

Listado de código 15: Entrenamiento de Clasificador Kernel Ridge Multiclase

1.6. Guía 3: Referencias y Documentos Históricos Obsoletos

Para propósitos de trazabilidad documental y mantenimiento de registros históricos, se reporta que el documento:

Informe de Desarrollo (v0.0.4): Preprocesamiento de Espectros FTIR y Clasificación Oncológica Celular mediante Espectroscopía Raman SERS (Matías Roberto Alemán, 20 de junio de 2026)

ha sido formalmente retirado de la línea activa de publicación y trasladado al directorio físico local:

`ftirlib_0.0.4/OBSOLETE/`

Cualquier consulta sobre la formulación experimental original, casos de estudio preliminares de óxido de grafeno (GO/rGO), pruebas de simulación física no colineal o validación de celdas tumorales primarias debe ser realizada consultando los archivos LaTeX e imágenes dentro de la carpeta mencionada. El presente ****Manual Extendido**** actúa como el documento oficial y consolidado que reemplaza y amplía todas las directrices técnicas del informe histórico.